

CADCAMWISE · SÉRIE TÉCNICA

# SQL Server na prática

## do NX ao banco de dados

Um minibook passo a passo baseado no caso real do PATHNC AUTOMATION: histórico de folhas de processo, Copilot de estratégias e orçamento — tudo em cima de um PathNCAutomationDB no SQL Server Express.

---

Diogenes · CADCAMWise

Setembro de 2026 · Edição 1

Stack: C# / .NET · ADO.NET · SQL Server Express · NXOpen

# Sumário

---

1. Por que um banco de dados no CAM
2. Vocabulário mínimo de SQL
3. Instalar, conectar e onde guardar a connection string
4. Passo 1 — Criar o schema (e evoluir sem quebrar)
5. Passo 2 — Gravar com transação: o Save to Database
6. Passo 3 — Ler e filtrar: views e busca de peças similares
7. Passo 4 — Guardar arquivos: HTML e miniaturas no banco
8. Passo 5 — Atualizar e calibrar: tempo real e upsert de configurações
9. Consultas analíticas que valem dinheiro
10. Erros que encontramos (e como cada um foi resolvido)
11. Boas práticas e próximos passos
12. Referências

# 1. Por que um banco de dados no CAM

Uma ferramentaria produz conhecimento todos os dias: qual estratégia funcionou em qual peça, qual ferramenta rodou a qual RPM, quanto tempo a máquina realmente levou. Esse conhecimento costuma morrer em pastas de arquivos `.prt` e na memória dos programadores. Um banco de dados relacional transforma isso em algo consultável.

No PATHNC AUTOMATION o caminho foi incremental, e é essa ordem que este livro segue:

- 1. **Shop Documentation** gera uma folha de processo em HTML a partir do setup CAM do NX.
- 2. O botão **Save to Database** passa a gravar a folha e as operações no **PathNCAutomationDB**, com revisão automática por peça.
- 3. Com histórico no banco, o **Copilot** encontra peças parecidas e sugere a estratégia real usada nelas.
- 4. Com tempo real registrado, o módulo de **orçamento** calibra estimativas e gera preço e prazo.

Cada etapa introduziu um conceito de SQL novo. Vamos vê-los na ordem em que apareceram.

## O modelo de dados final

Tudo gira em torno de duas tabelas — a folha ( `ShopDoc` ) e suas operações ( `ShopDocOperation` ) — e duas tabelas satélite para orçamento.

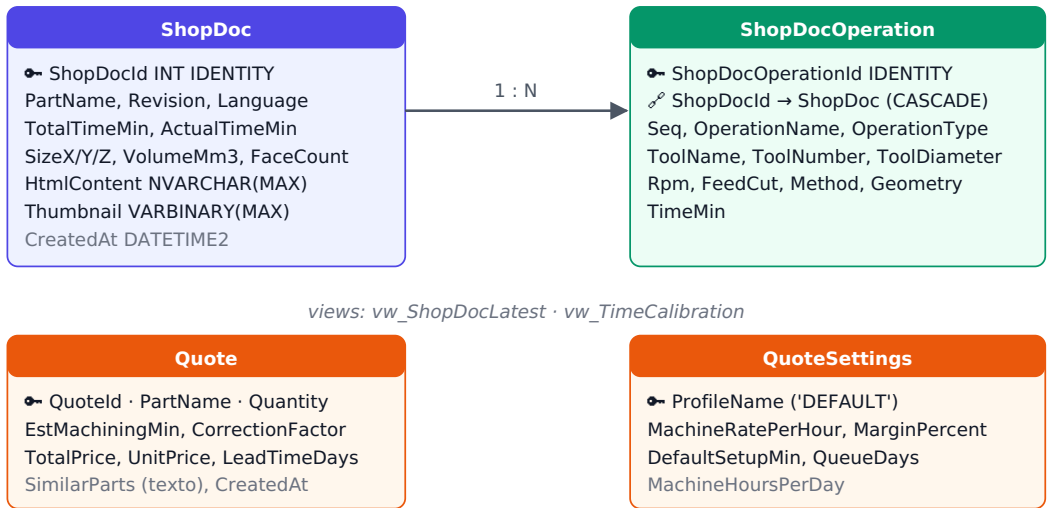


Figura 1 — Modelo de dados do PathNCAutomationDB (módulo Shop Doc / Copilot / Quote).

## 2. Vocabulário mínimo de SQL

SQL é uma linguagem declarativa: você diz *o que* quer, não *como* buscar. Os termos abaixo cobrem tudo o que este livro usa.

| Termo                  | O que é                                                       | Onde aparece no PATHNC                        |
|------------------------|---------------------------------------------------------------|-----------------------------------------------|
| Tabela                 | Conjunto de linhas com colunas tipadas                        | ShopDoc , ShopDocOperation                    |
| Chave primária (PK)    | Coluna que identifica a linha; IDENTITY gera o número sozinho | ShopDocId INT IDENTITY PRIMARY KEY            |
| Chave estrangeira (FK) | Coluna que aponta para a PK de outra tabela                   | ShopDocOperation.ShopDocId REFERENCES ShopDoc |
| ON DELETE CASCADE      | Apagar o pai apaga os filhos                                  | Apagar uma folha apaga suas operações         |
| NULL                   | Ausência de valor (não é zero, não é vazio)                   | RPM de uma operação sem parâmetro             |
| Índice                 | Estrutura auxiliar que acelera buscas por uma coluna          | IX_ShopDoc_PartName                           |
| View                   | Consulta salva que se comporta como tabela                    | vw_ShopDocLatest                              |
| Transação              | Grupo de comandos que vale tudo ou nada                       | Cabeçalho + operações no Save                 |
| Parâmetro              | Valor passado separado do texto SQL                           | @PartName em todo comando                     |

### Tipos de dados usados

| Tipo           | Uso                            | Exemplo                            |
|----------------|--------------------------------|------------------------------------|
| INT            | Inteiros, IDs, contadores      | Revision , ToolNumber              |
| FLOAT          | Números reais                  | Rpm , TotalTimeMin                 |
| NVARCHAR(n)    | Texto Unicode até n caracteres | PartName NVARCHAR(200)             |
| NVARCHAR(MAX)  | Texto longo (até 2 GB)         | HtmlContent                        |
| VARBINARY(MAX) | Bytes (imagens, arquivos)      | Thumbnail                          |
| DATETIME2      | Data e hora com precisão alta  | CreatedAt DEFAULT SYSUTCDATETIME() |

**Por que NVARCHAR e não VARCHAR?** O N indica Unicode. O Shop Doc gera folhas em oito idiomas, inclusive chinês — com VARCHAR os caracteres virariam ? .

### O mesmo conceito em outros bancos

SQL é padronizado, mas cada motor tem dialeto. Se um dia o PATHNC precisar rodar em SQLite (portátil, sem servidor) ou PostgreSQL (nuvem), estas são as diferenças que aparecem primeiro:

| Conceito            | SQL Server                      | SQLite                            | PostgreSQL                            | MySQL                   |
|---------------------|---------------------------------|-----------------------------------|---------------------------------------|-------------------------|
| Auto-incremento     | INT IDENTITY                    | INTEGER PRIMARY KEY AUTOINCREMENT | SERIAL / GENERATED ALWAYS AS IDENTITY | INT AUTO_INCREMENT      |
| Texto longo         | NVARCHAR(MAX)                   | TEXT                              | TEXT                                  | LONGTEXT                |
| Bytes               | VARBINARY(MAX)                  | BLOB                              | BYTEA                                 | LONGBLOB                |
| Limitar linhas      | SELECT TOP 10                   | LIMIT 10                          | LIMIT 10                              | LIMIT 10                |
| Pegar ID gerado     | OUTPUT INSERTED.Id              | last_insert_rowid()               | RETURNING id                          | LAST_INSERT_ID()        |
| Data/hora atual UTC | SYSUTCDATETIME()                | datetime('now')                   | now() at time zone 'utc'              | UTC_TIMESTAMP()         |
| Upsert              | MERGE ou UPDATE + IF @@ROWCOUNT | INSERT ... ON CONFLICT            | INSERT ... ON CONFLICT                | ON DUPLICATE KEY UPDATE |

## 3. Instalar, conectar e onde guardar a connection string

### 3.1 O servidor

O PATHNC usa o **SQL Server Express**, gratuito, instalado na máquina do programador como instância nomeada `.\SQLEXPRESS`. Duas ferramentas acompanham:

- **SSMS** (SQL Server Management Studio) — interface gráfica para rodar scripts e olhar tabelas.
- **sqlcmd** — linha de comando; ideal para scripts e automação.

```
Prompt de comando - sqlcmd

:: Listar os bancos da instancia (foi assim que confirmamos onde o banco estava)
sqlcmd -S ".\SQLEXPRESS" -Q "SELECT name FROM sys.databases"

:: Rodar um script de schema no banco certo (-d e obrigatorio, senao vai para master)
sqlcmd -S ".\SQLEXPRESS" -d PathNCAutomationDB -i "C:\scripts\ShopDoc_Schema.sql"

:: Conferir tabelas criadas
sqlcmd -S ".\SQLEXPRESS" -d PathNCAutomationDB -Q "SELECT name FROM sys.tables WHERE name LIKE 'ShopDoc%'"
```

### 3.2 A connection string

É a frase que diz ao .NET onde está o banco e como autenticar. No nosso caso:

```
PathNCAutomationDB.connection

Server=.\SQLEXPRESS;Database=PathNCAutomationDB;Integrated Security=true;
```

| Parte                       | Significado                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------|
| Server=.\SQLEXPRESS         | Máquina local ( . ), instância SQLEXPRESS . Poderia ser SERVIDOR\SQLEXPRESS ou (localdb)\MSSQLLocalDB . |
| Database=PathNCAutomationDB | Banco padrão da conexão.                                                                                |
| Integrated Security=true    | Autentica com o usuário do Windows — sem senha no arquivo.                                              |

### 3.3 Onde guardar

O PATHNC roda como DLL *dentro do processo do NX*. Isso muda uma coisa importante: o `App.config` do seu executável nunca é lido, porque o executável é o `ugraf.exe`. Por isso o resolvidor procura em três lugares, nesta ordem:

```
ShopDocDatabase.cs - ShopDocRepository.ResolverConnectionString

// Ordem de resolucao (sem depender de System.Configuration):
// 1) variavel de ambiente PATHNC_DB_CONNECTION
// 2) arquivo PathNCAutomationDB.connection ao lado da DLL
// 3) LocalDB padrao
public static string ResolverConnectionString()
{
    string env = Environment.GetEnvironmentVariable("PATHNC_DB_CONNECTION");
    if (!string.IsNullOrEmpty(env))
        return env.Trim();

    string dir = System.IO.Path.GetDirectoryName(
        System.Reflection.Assembly.GetExecutingAssembly().Location);
    string arquivo = System.IO.Path.Combine(dir, "PathNCAutomationDB.connection");
    if (File.Exists(arquivo))
        return File.ReadAllText(arquivo).Trim();

    return @"Server=(localdb)\MSSQLLocalDB;Database=PathNCAutomationDB;Integrated Security=true;";
}
```

**O erro que vimos:** *Cannot open database "PathNCAutomationDB" requested by the login. Login failed for user DIOGENESCSF\dcard.* Tradução: a conexão chegou num servidor (o LocalDB do fallback), mas o banco não existia lá. A solução foi criar o arquivo `.connection` apontando para `.\SQLEXPRESS`, onde o banco realmente estava.

### 3.4 Abrir e fechar a conexão

No ADO.NET a conexão vive dentro de um `using`: abre, executa, fecha — mesmo se der exceção. Não existe conexão "aberta o tempo todo"; o pool do .NET reaproveita conexões por baixo dos panos.

```
● ● ● Padrão básico - SqlConnection + SqlCommand

using (SqlConnection cn = new SqlConnection(_connectionString))
using (SqlCommand cmd = new SqlCommand("SELECT COUNT(*) FROM dbo.ShopDoc", cn))
{
    cn.Open();
    int total = (int)cmd.ExecuteScalar(); // ExecuteScalar: primeira coluna da primeira linha
}
```

### 3.5 Alternativas para comparar

| Abordagem                          | Prós                                                                       | Contras                                                 | Quando usar                                        |
|------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------|----------------------------------------------------|
| <b>ADO.NET puro</b> (o que usamos) | Sem dependências, controle total, roda dentro do NX sem conflito de versão | Mais código repetitivo (parâmetros, leitura de colunas) | Plug-ins, DLLs que rodam em processos de terceiros |
| <b>Dapper</b>                      | Mapeia linhas para objetos em uma linha; mesmo SQL                         | Pacote NuGet a carregar dentro do NX                    | Apps próprios com muitas consultas                 |
| <b>Entity Framework Core</b>       | Modelo em C#, migrations automáticas, LINQ                                 | Pesado para carregar no NX; SQL gerado é opaco          | APIs web, o futuro Shop Doc na web                 |

## 4. Passo 1 — Criar o schema (e evoluir sem quebrar)

### 4.1 A tabela pai e a tabela filha

```
ShopDoc_Schema.sql (resumido)

CREATE TABLE dbo.ShopDoc (
  ShopDocId      INT IDENTITY PRIMARY KEY,
  PartName       NVARCHAR(200) NOT NULL,
  Revision       INT          NOT NULL DEFAULT 1,
  TotalTimeMin   FLOAT        NULL,
  HtmlContent    NVARCHAR(MAX) NULL,
  CreatedAt      DATETIME2    NOT NULL DEFAULT SYSUTCDATETIME()
);
CREATE INDEX IX_ShopDoc_PartName ON dbo.ShopDoc(PartName, CreatedAt DESC);

CREATE TABLE dbo.ShopDocOperation (
  ShopDocOperationId INT IDENTITY PRIMARY KEY,
  ShopDocId          INT NOT NULL REFERENCES dbo.ShopDoc(ShopDocId) ON DELETE CASCADE,
  Seq               INT NOT NULL,
  OperationName     NVARCHAR(100) NOT NULL,
  ToolName          NVARCHAR(100) NULL,
  Rpm               FLOAT NULL,
  FeedCut           FLOAT NULL,
  TimeMin           FLOAT NULL
);
```

Três decisões nesse script valem comentário:

- **IDENTITY** — o banco numera as linhas; o C# nunca inventa IDs.
- **ON DELETE CASCADE** — apagar uma folha apaga as operações dela. Sem isso o `DELETE` falharia por violar a FK.
- **Índice em (PartName, CreatedAt DESC)** — a consulta mais comum é "última folha desta peça"; o índice faz ela ser instantânea mesmo com milhares de linhas.

### 4.2 Scripts idempotentes: o erro que nos ensinou isso

Na primeira execução, a tabela `ShopDoc` já existia com uma versão antiga das colunas. O script tinha `IF OBJECT_ID('dbo.ShopDoc') IS NULL` — então pulou silenciosamente a criação, e o C# quebrou com *Invalid column name 'Language', 'MachineName', 'OperationCount'*.

A lição: scripts de banco devem poder rodar **várias vezes** sem estragar nada, e devem **adicionar** o que falta em vez de tentar recriar. É o que `COL_LENGTH` permite:

```
ShopDoc_Migracao.sql

-- Migracao idempotente: adiciona cada coluna so se ela ainda nao existir
IF COL_LENGTH('dbo.ShopDoc', 'Language') IS NULL
  ALTER TABLE dbo.ShopDoc ADD Language NVARCHAR(10) NULL;
IF COL_LENGTH('dbo.ShopDoc', 'MachineName') IS NULL
  ALTER TABLE dbo.ShopDoc ADD MachineName NVARCHAR(100) NULL;
IF COL_LENGTH('dbo.ShopDoc', 'OperationCount') IS NULL
  ALTER TABLE dbo.ShopDoc ADD OperationCount INT NOT NULL DEFAULT 0;

-- Confirmacao: lista as colunas reais
SELECT c.name FROM sys.columns c
WHERE c.object_id = OBJECT_ID('dbo.ShopDoc') ORDER BY c.column_id;
```

**Regra prática:** numere as migrações (`Migracao_1, _2, _3...`) e nunca edite uma que já rodou em produção. Cada nova necessidade vira um arquivo novo. O EF Core faz exatamente isso, só que gerando os arquivos para você.

### 4.3 Views: consultas com nome

A view `vw_ShopDocLatest` resolve uma pergunta que se repete: "a última revisão de cada peça, só as que têm assinatura geométrica".

vw\_ShopDocLatest

```
CREATE VIEW dbo.vw_ShopDocLatest AS
SELECT s.*
FROM dbo.ShopDoc s
JOIN (
    SELECT PartName, MAX(Revision) AS MaxRev
    FROM dbo.ShopDoc
    GROUP BY PartName
) u ON u.PartName = s.PartName AND u.MaxRev = s.Revision
WHERE s.SizeX IS NOT NULL;
```

A subconsulta agrupa por peça e pega a maior revisão; o `JOIN` traz a linha completa daquela revisão. Uma forma equivalente, mais moderna, usa *window function*:

Alternativa com window function

```
-- Mesmo resultado com ROW_NUMBER(): numera as revisoes de cada peça da maior para a menor
SELECT * FROM (
    SELECT s.*, ROW_NUMBER() OVER (PARTITION BY PartName ORDER BY Revision DESC) AS rn
    FROM dbo.ShopDoc s
) x
WHERE rn = 1 AND SizeX IS NOT NULL;
```

**Detalhe de SQL Server:** uma view com `SELECT s.*` guarda a lista de colunas no momento da criação. Quando adicionamos `Thumbnail`, recriamos a view ( `DROP VIEW + CREATE VIEW` ) — senão a coluna nova não aparece.

## 5. Passo 2 — Gravar com transação: o Save to Database

O botão precisa gravar um cabeçalho e N operações. Se a terceira operação falhar, não pode sobrar um cabeçalho pela metade. Isso é o que uma **transação** garante.

```
ShopDocRepository.Save - transação

public Tuple<int, int> Save(ShopDocModel doc)
{
    using (SqlConnection cn = new SqlConnection(_connectionString))
    {
        cn.Open();
        using (SqlTransaction tx = cn.BeginTransaction())
        {
            try
            {
                int revision = ProximaRevisao(cn, tx, doc.PartName);
                int shopDocId = InserirCabecalho(cn, tx, doc, revision);
                InserirOperacoes(cn, tx, shopDocId, doc.Operations);
                tx.Commit(); // tudo valeu
                return Tuple.Create(shopDocId, revision);
            }
            catch
            {
                tx.Rollback(); // nada valeu
                throw;
            }
        }
    }
}
```

### 5.1 Revisão automática

Em vez de sobrescrever, cada Save da mesma peça vira uma revisão nova. É uma consulta de agregação dentro da mesma transação:

```
Próxima revisão

SELECT ISNULL(MAX(Revision), 0) + 1 FROM dbo.ShopDoc WHERE PartName = @p
```

`ISNULL(x, 0)` trata o caso da primeira folha da peça (quando `MAX` devolve `NULL`).

### 5.2 Inserir e recuperar o ID gerado

```
InserirCabecalho - OUTPUT INSERTED e parâmetros

const string sql = @"
INSERT INTO dbo.ShopDoc (PartName, PartPath, Revision, Language, TotalTimeMin, HtmlContent)
OUTPUT INSERTED.ShopDocId
VALUES (@PartName, @PartPath, @Revision, @Language, @TotalTimeMin, @HtmlContent);";

using (SqlCommand cmd = new SqlCommand(sql, cn, tx))
{
    cmd.Parameters.AddWithValue("@PartName", doc.PartName);
    cmd.Parameters.AddWithValue("@PartPath", (object)doc.PartPath ?? DBNull.Value);
    cmd.Parameters.AddWithValue("@Revision", revision);
    cmd.Parameters.AddWithValue("@Language", (object)doc.Language ?? DBNull.Value);
    cmd.Parameters.AddWithValue("@TotalTimeMin", doc.TotalTimeMin);
    cmd.Parameters.Add("@HtmlContent", SqlDbType.NVarChar, -1).Value =
        (object)doc.HtmlContent ?? DBNull.Value; // -1 = MAX
    return (int)cmd.ExecuteScalar(); // o OUTPUT vira o resultado
}
```

Três detalhes:

- **Parâmetros sempre.** Nunca monte SQL concatenando texto do usuário ( `"WHERE PartName = '" + nome + "'"` ). Um nome de peça com apóstrofo já quebraria; um malicioso apagaria a tabela. Parâmetros também deixam o servidor reaproveitar o plano de execução.
- `DBNull.Value` é como o C# manda `NULL`. Um `null` de C# em `AddWithValue` dá erro.
- `OUTPUT INSERTED.ShopDocId` devolve o ID gerado na mesma ida ao servidor. A alternativa antiga era um segundo comando `SELECT SCOPE_IDENTITY()`.

### 5.3 Inserir muitas linhas: reaproveite o comando

Para as operações, o comando é preparado uma vez e só os valores dos parâmetros mudam a cada linha. É muito mais rápido do que criar um `SqlCommand` por linha.

```
●●● InserirOperacoes – parâmetros tipados reutilizados

using (SqlCommand cmd = new SqlCommand(sql, cn, tx))
{
    cmd.Parameters.Add("@ShopDocId", SqlDbType.Int);
    cmd.Parameters.Add("@Seq", SqlDbType.Int);
    cmd.Parameters.Add("@OperationName", SqlDbType.NVarChar, 100);
    cmd.Parameters.Add("@Rpm", SqlDbType.Float);
    // ...

    foreach (ShopDocOperationModel o in ops)
    {
        cmd.Parameters["@ShopDocId"].Value = shopDocId;
        cmd.Parameters["@Seq"].Value = o.Seq;
        cmd.Parameters["@OperationName"].Value = o.OperationName;
        cmd.Parameters["@Rpm"].Value = (object)o.Rpm ?? DBNull.Value;
        cmd.ExecuteNonQuery();    // ExecuteNonQuery: comando sem resultado (INSERT/UPDATE/DELETE)
    }
}
```

| Método                         | Devolve                    | Use para                    |
|--------------------------------|----------------------------|-----------------------------|
| <code>ExecuteNonQuery()</code> | Linhas afetadas (int)      | INSERT, UPDATE, DELETE      |
| <code>ExecuteScalar()</code>   | Primeira célula (object)   | COUNT, MAX, OUTPUT INSERTED |
| <code>ExecuteReader()</code>   | <code>SqlDataReader</code> | SELECT com várias linhas    |

**Para volumes grandes** (milhares de linhas por save), o SQL Server tem `SqlBulkCopy` — copia um `DataTable` inteiro de uma vez. Para as 10-50 operações de uma folha, o loop com parâmetros reaproveitados é mais que suficiente.

## 6. Passo 3 — Ler e filtrar: views e busca de peças similares

O Copilot precisa de duas leituras: os candidatos (última revisão de cada peça, com assinatura) e as operações de cada vizinha escolhida.

```
StrategyHistoryService.CarregarCandidatos - SqlDataReader

private List<SimilarPart> CarregarCandidatos(string partNameAtual)
{
    const string sql = @"
SELECT ShopDocId, PartName, Revision, SizeX, SizeY, SizeZ, VolumeMm3,
       ISNULL(FaceCount, 0), ISNULL(TotalTimeMin, 0), CreatedAt, Thumbnail
FROM dbo.vw_ShopDocLatest
WHERE PartName <> @atual;           // nunca compara a peça com ela mesma

List<SimilarPart> lista = new List<SimilarPart>();
using (SqlConnection cn = new SqlConnection(_cs))
using (SqlCommand cmd = new SqlCommand(sql, cn))
{
    cmd.Parameters.AddWithValue("@atual", partNameAtual ?? "");
    cn.Open();
    using (SqlDataReader r = cmd.ExecuteReader())
    {
        while (r.Read())
        {
            lista.Add(new SimilarPart
            {
                ShopDocId = r.GetInt32(0),
                PartName = r.GetString(1),
                SizeX = r.GetDouble(3),
                VolumeMm3 = r.IsDBNull(6) ? 0 : r.GetDouble(6), // NULL -> 0
                Thumbnail = r.IsDBNull(10) ? null : (byte[])r[10],
                // ...
            });
        }
    }
}
return lista;
}
```

Regras de leitura que evitam exceções:

- `r.GetDouble(i)` em coluna `NULL` lança exceção — teste `r.IsDBNull(i)` antes, ou use `ISNULL()` no SQL para nunca receber `NULL`.
- Os índices (0, 1, 2...) seguem a ordem do `SELECT`. Se você trocar a ordem das colunas, troque os índices — ou use `r["PartName"]` por nome, um pouco mais lento e mais legível.
- Bytes vêm como `(byte[])r[i]`.

### 6.1 Onde a lógica mora: SQL ou C#?

A similaridade (dimensões ordenadas, volume, faces) é calculada em C#, não em SQL. A escolha é deliberada: com dezenas ou centenas de peças, trazer todos os candidatos e pontuar na aplicação é simples e rápido; com centenas de milhares, valeria mover o cálculo para uma *stored procedure* ou pré-filtrar por faixa de dimensão no `WHERE`:

```
Pré-filtro por faixa

-- Pre-filtro opcional para bases grandes: so peças com dimensoes ate 2x diferentes
SELECT ... FROM dbo.vw_ShopDocLatest
WHERE PartName <> @atual
      AND SizeX BETWEEN @sx * 0.5 AND @sx * 2.0
      AND SizeY BETWEEN @sy * 0.5 AND @sy * 2.0;
```

### 6.2 Consulta com filtro opcional (Quote History)

O histórico de orçamentos tem um campo de busca que pode ficar vazio. O truque `(@f IS NULL OR ... LIKE @f)` resolve os dois casos num único SQL:

```
QuoteService.ListQuotes

SELECT TOP (@max) QuoteId, PartName, CustomerName, TotalPrice, LeadTimeDays, CreatedAt
FROM dbo.Quote
WHERE (@f IS NULL OR PartName LIKE @f OR CustomerName LIKE @f)
ORDER BY CreatedAt DESC;
```

No C#, o parâmetro recebe `"%" + filtro + "%"` quando há texto, e `DBNull.Value` quando não há. `LIKE '%PLACA%'` é "contém"; `LIKE 'PLACA%'` é "começa com" (e usa índice).

# 7. Passo 4 — Guardar arquivos: HTML e miniaturas no banco

Duas coisas "não tabulares" entraram no banco: a folha HTML inteira e uma miniatura PNG da peça. Cada uma com um tipo próprio.

| Conteúdo      | Coluna      | Tipo SQL       | Tipo C# | Parâmetro               |
|---------------|-------------|----------------|---------|-------------------------|
| HTML da folha | HtmlContent | NVARCHAR(MAX)  | string  | SqlDbType.NVarChar, -1  |
| Miniatura     | Thumbnail   | VARBINARY(MAX) | byte[]  | SqlDbType.VarBinary, -1 |

Gravando bytes – o -1 significa MAX

```
cmd.Parameters.Add("@Thumbnail", SqlDbType.VarBinary, -1).Value =
(object)doc.Thumbnail ?? DBNull.Value;
```

A miniatura é gerada no NX (vista isométrica, `ImageExportBuilder`), reduzida para 480 px de largura com `System.Drawing` e serializada em PNG. Uns 30-60 KB por peça — irrelevante para o banco.

ShopDocCollector.CapturarThumbnail (trecho)

```
using (Bitmap reduzida = new Bitmap(w, h))
using (Graphics g = Graphics.FromImage(reduzida))
{
    g.InterpolationMode = InterpolationMode.HighQualityBicubic;
    g.DrawImage(original, 0, 0, w, h);
    using (MemoryStream ms = new MemoryStream())
    {
        reduzida.Save(ms, ImageFormat.Png);
        return ms.ToArray(); // -> byte[] pronto para o VARBINARY
    }
}
```

## 7.1 Conferindo pelo SQL

DATALENGTH

```
-- Tamanho em bytes de cada arquivo guardado
SELECT ShopDocId, PartName, Revision,
       DATALENGTH(HtmlContent) AS HtmlBytes,
       DATALENGTH(Thumbnail) AS ThumbBytes
FROM dbo.ShopDoc ORDER BY ShopDocId DESC;
```

**O caso da imagem que não apareceu:** a peça similar veio sem miniatura porque tinha sido salva antes da migração 3. `DATALENGTH(Thumbnail)` devolveu NULL. Salvar de novo criou uma revisão com a foto — e a view `vw_ShopDocLatest`, por pegar sempre a última revisão, passou a mostrá-la.

## 7.2 Arquivo no banco ou caminho no banco?

| Estratégia                                | Prós                                                                       | Contras                                           | Veredito no PATHNC                                                          |
|-------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Bytes no banco</b><br>(nosso caso)     | Backup único; a folha "viaja" com o registro; nada quebra se a pasta mudar | Banco cresce; ruim para arquivos de dezenas de MB | Certo para HTML e miniaturas pequenas                                       |
| <b>Caminho no banco, arquivo no disco</b> | Banco leve                                                                 | Arquivo some, link quebra; backup em dois lugares | Faz sentido para os <code>.prt</code> — e é o que <code>PartPath</code> faz |
| <b>FILESTREAM</b> (SQL Server)            | Melhor dos dois: bytes gerenciados pelo banco, guardados no NTFS           | Configuração extra na instância                   | Se um dia os HTMLs com imagens embutidas passarem de alguns MB              |

## 8. Passo 5 — Atualizar e calibrar: tempo real e upsert

### 8.1 UPDATE com subconsulta

Registrar o tempo real atualiza a **última revisão** da peça. O alvo é resolvido dentro do próprio `WHERE` :

```
QuoteService.RegisterActualTime

UPDATE dbo.ShopDoc
SET ActualTimeMin = @t, ActualSetupMin = @s, ActualTimeNote = @n
WHERE ShopDocId = (
    SELECT TOP 1 ShopDocId FROM dbo.ShopDoc
    WHERE PartName = @p ORDER BY Revision DESC
);
```

O C# usa o retorno de `ExecuteNonQuery()` — se for 0, a peça não existia, e o diálogo avisa.

### 8.2 Upsert: atualizar ou inserir

Os parâmetros de custo têm um perfil por nome. "Salvar" precisa atualizar se o perfil existe e criar se não existe. O padrão mais legível no SQL Server:

```
QuoteService.SaveSettings – UPDATE + IF @@ROWCOUNT

UPDATE dbo.QuoteSettings
SET MachineRatePerHour = @m, MarginPercent = @mg, UpdatedAt = SYSUTCDATETIME()
WHERE ProfileName = @p;

IF @@ROWCOUNT = 0
    INSERT INTO dbo.QuoteSettings (ProfileName, MachineRatePerHour, MarginPercent)
    VALUES (@p, @m, @mg);
```

`@@ROWCOUNT` é quantas linhas o comando anterior afetou. A alternativa "oficial" é o `MERGE`, mais poderoso e mais difícil de ler:

```
Mesma coisa com MERGE

MERGE dbo.QuoteSettings AS alvo
USING (SELECT @p AS ProfileName, @m AS MachineRatePerHour, @mg AS MarginPercent) AS fonte
ON alvo.ProfileName = fonte.ProfileName
WHEN MATCHED THEN UPDATE SET MachineRatePerHour = fonte.MachineRatePerHour, MarginPercent = fonte.MarginPercent
WHEN NOT MATCHED THEN INSERT (ProfileName, MachineRatePerHour, MarginPercent)
VALUES (fonte.ProfileName, fonte.MachineRatePerHour, fonte.MarginPercent);
```

### 8.3 Uma view para calibrar estimativas

O fator de correção do orçamento é a mediana de *real ÷ estimado*. A view expõe a razão já calculada, protegendo a divisão por zero com `CASE` :

```
vw_TimeCalibration

CREATE VIEW dbo.vw_TimeCalibration AS
SELECT ShopDocId, PartName, Revision,
    TotalTimeMin AS EstimatedMin, ActualTimeMin,
    CASE WHEN TotalTimeMin > 0 THEN ActualTimeMin / TotalTimeMin END AS Ratio,
    CreatedAt
FROM dbo.ShopDoc
WHERE ActualTimeMin IS NOT NULL AND TotalTimeMin IS NOT NULL AND TotalTimeMin > 0;
```

A mediana é calculada em C# (ordenar e pegar o meio). O SQL Server também sabe fazer, com `PERCENTILE_CONT` :

```
Mediana no próprio SQL

SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Ratio) OVER () AS MedianaRatio
FROM dbo.vw_TimeCalibration;
```

## 9. Consultas analíticas que valem dinheiro

Com o histórico gravado, estas consultas respondem perguntas que antes exigiam abrir peça por peça no NX. Todas rodam direto no SSMS.

### 9.1 Tempo médio por tipo de operação

GROUP BY + agregações

```
SELECT OperationType, COUNT(*) AS Qtd,
       AVG(TimeMin) AS MediaMin, MAX(TimeMin) AS MaxMin
FROM dbo.ShopDocOperation
WHERE TimeMin > 0
GROUP BY OperationType
ORDER BY MediaMin DESC;
```

### 9.2 Ferramentas com parâmetros inconsistentes

HAVING filtra *depois* de agrupar — aqui, só ferramentas cujo RPM varia mais de 20% entre programas:

HAVING

```
SELECT ToolName,
       MIN(Rpm) AS RpmMin, MAX(Rpm) AS RpmMax,
       MIN(FeedCut) AS FeedMin, MAX(FeedCut) AS FeedMax,
       COUNT(DISTINCT ShopDocId) AS Programas
FROM dbo.ShopDocOperation
WHERE Rpm IS NOT NULL AND Rpm > 0
GROUP BY ToolName
HAVING MAX(Rpm) > MIN(Rpm) * 1.2
ORDER BY Programas DESC;
```

### 9.3 Ferramentas da biblioteca que nunca saem em programa

Um LEFT JOIN mantém todas as ferramentas da biblioteca; as que não têm correspondência nas operações aparecem com NULL do lado direito:

LEFT JOIN ... WHERE ... IS NULL (anti-join)

```
SELECT t.ToolName, t.Diameter
FROM dbo.Tools t
LEFT JOIN dbo.ShopDocOperation o ON o.ToolName = t.ToolName
WHERE o.ShopDocOperationId IS NULL
ORDER BY t.ToolName;
```

### 9.4 O que mudou entre duas revisões

CTE (WITH) + FULL OUTER JOIN – diff de revisões

```
DECLARE @peca NVARCHAR(200) = 'PLACA_P1';
WITH r AS (
    SELECT o.OperationName, o.ToolName, o.Rpm, o.FeedCut, o.TimeMin, s.Revision
    FROM dbo.ShopDocOperation o JOIN dbo.ShopDoc s ON s.ShopDocId = o.ShopDocId
    WHERE s.PartName = @peca AND s.Revision IN (1, 2)
)
SELECT COALESCE(a.OperationName, b.OperationName) AS Operacao,
       a.ToolName AS Ferr_R1, b.ToolName AS Ferr_R2,
       a.Rpm AS Rpm_R1, b.Rpm AS Rpm_R2,
       a.TimeMin AS Min_R1, b.TimeMin AS Min_R2
FROM (SELECT * FROM r WHERE Revision = 1) a
FULL OUTER JOIN (SELECT * FROM r WHERE Revision = 2) b ON a.OperationName = b.OperationName
ORDER BY Operacao;
```

Linhas com NULL só de um lado são operações adicionadas ou removidas; as demais mostram lado a lado o que mudou.

### 9.5 Ranking por janela

● ● ● RANK() OVER (PARTITION BY ...)

-- As 3 operacoes mais longas de CADA peca

```
SELECT * FROM (  
    SELECT s.PartName, o.OperationName, o.TimeMin,  
           RANK() OVER (PARTITION BY s.PartName ORDER BY o.TimeMin DESC) AS Pos  
    FROM dbo.ShopDocOperation o JOIN dbo.ShopDoc s ON s.ShopDocId = o.ShopDocId  
  ) x WHERE Pos <= 3;
```

# 10. Erros que encontramos (e como cada um foi resolvido)

Nenhum desses erros era "de SQL" no sentido estrito — todos eram de integração. É assim na prática.

| Mensagem                                                                        | Causa real                                                                             | Solução                                                              |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| 'Path' is an ambiguous reference between 'NXOpen.CAM.Path' and 'System.IO.Path' | O using NXOpen.CAM traz uma classe chamada Path (toolpath)                             | Qualificar: System.IO.Path.Combine(...)                              |
| The name 'ConfigurationManager' does not exist                                  | Faltava referência a System.Configuration — e o App.config nem seria lido dentro do NX | Trocar por arquivo .connection ao lado da DLL + variável de ambiente |
| Unable to load DLL 'libugini'                                                   | O programa foi executado como EXE fora do NX; o NXOpen não acha as bibliotecas nativas | Rodar pelo NX (Tools → Journal → Play). Não era erro de banco        |
| Cannot open database "PathNCAutomationDB" requested by the login                | Conectou no LocalDB (fallback) onde o banco não existe                                 | Criar PathNCAutomationDB.connection com Server=.\SQLEXPRESS          |
| Invalid column name 'Language', 'MachineName'...                                | Tabela já existia com schema antigo; o IF OBJECT_ID IS NULL pulou a criação            | Migração idempotente com COL_LENGTH + ALTER TABLE ADD                |
| No similar parts found (com peças salvas)                                       | Registros anteriores à migração não tinham assinatura; a view os ignora                | Salvar de novo (nova revisão com SizeX preenchido)                   |
| Miniatura da peça similar não aparece                                           | Thumbnail NULL nas revisões antigas                                                    | DATALength(Thumbnail) para confirmar; salvar de novo                 |
| Cannot implicitly convert NCGroup to Tool                                       | CreateTool devolve a classe base                                                       | Cast explícito (NXOpen.CAM.Tool)                                     |

**Método de diagnóstico que funcionou todas as vezes:** antes de mexer no C#, rode a consulta equivalente no SSMS. Se o SQL devolve o que você espera, o problema está na aplicação; se não devolve, está no banco ou nos dados. Metade dos erros acima foi resolvida com um SELECT de uma linha.

# 11. Boas práticas e próximos passos

## Checklist que o projeto segue

- ✓ Parâmetros em todo comando; nunca concatenar texto no SQL.
- ✓ Conexão dentro de `using` ; abrir tarde, fechar cedo.
- ✓ Escritas com múltiplos comandos dentro de uma transação.
- ✓ Scripts de schema idempotentes e numerados; nunca editar migração já aplicada.
- ✓ `NULL` tratado dos dois lados: `DBNull.Value` ao gravar, `IsDBNull` / `ISNULL` ao ler.
- ✓ Views para consultas que se repetem; C# para lógica que muda com frequência.
- ✓ Índice na combinação de colunas do `WHERE` + `ORDER BY` mais usada.
- ✓ Connection string fora do código-fonte e fora do App.config quando a DLL roda em processo de terceiro.
- ✓ Uma consulta no SSMS antes de qualquer alteração no C#.

## Backup — o item que ninguém lembra até precisar

●●● Backup e restore (SSMS ou sqlcmd)

```
BACKUP DATABASE PathNCAutomationDB
TO DISK = 'D:\Backups\PathNCAutomationDB.bak'
WITH INIT, COMPRESSION;

-- Restaurar em outra máquina
RESTORE DATABASE PathNCAutomationDB FROM DISK = 'D:\Backups\PathNCAutomationDB.bak' WITH REPLACE;
```

Agende com o Agendador de Tarefas do Windows chamando `sqlcmd -S .\SQLEXPRESS -Q "BACKUP DATABASE ..." toda noite`. O Express não tem SQL Agent; o Agendador substitui.

## O que vem depois

| Próxima frente                             | Conceito SQL novo                                                          |
|--------------------------------------------|----------------------------------------------------------------------------|
| Alerta de parâmetro fora do padrão no Save | Mediana com <code>PERCENTILE_CONT</code> , comparação por janela           |
| Diff de revisões no painel                 | CTE + <code>FULL OUTER JOIN</code> (seção 9.4)                             |
| Dashboard de ferramentas                   | Anti-join, <code>SUM</code> por ferramenta, <code>TOP N</code>             |
| Shop Doc na web                            | API lendo <code>NVARCHAR(MAX)</code> ; imagens embutidas em base64 no HTML |
| Multi-cliente na nuvem                     | Azure SQL, EF Core migrations, tenant por coluna                           |

## 12. Referências

---

- Microsoft Learn — *SQL Server Express*: instalação e limites (10 GB por banco, 1 socket/4 núcleos, 1 GB RAM).
- Microsoft Learn — *Transact-SQL reference*: `CREATE TABLE`, `ALTER TABLE`, `MERGE`, `OUTPUT`, window functions, `PERCENTILE_CONT`.
- Microsoft Learn — *ADO.NET: SqlConnection, SqlCommand, SqlDataReader, SqlTransaction, SqlBulkCopy*.
- Microsoft Learn — *Connection strings e SqlConnectionStringBuilder*.
- Microsoft Learn — *sqlcmd utility*.
- Itzik Ben-Gan, *T-SQL Fundamentals* (Microsoft Press) — o melhor livro para ir além do básico em SQL Server.
- C. J. Date, *SQL and Relational Theory* — para entender o porquê do modelo relacional.
- Dapper ([github.com/DapperLib/Dapper](https://github.com/DapperLib/Dapper)) e EF Core ([learn.microsoft.com/ef/core](https://learn.microsoft.com/ef/core)) — as alternativas comparadas no capítulo 3.
- Siemens NX Open — *NXOpen.CAM*, *UFSession.Param* (leitura de RPM/avanço), *ImageExportBuilder*.

---

Todo o código deste livro foi extraído do projeto PATHNC AUTOMATION (CADCAMWise) durante o desenvolvimento dos módulos Shop Documentation, Copilot e Quote, em setembro de 2026. Os trechos foram encurtados para caber na página; os arquivos completos são `ShopDocDatabase.cs`, `StrategyHistoryService.cs`, `StrategyApplier.cs`, `QuoteService.cs` e os scripts `ShopDoc_Schema.sql` e `ShopDoc_Migracao_1..4.sql`.